```
Output:
string(3) "123"
int(123)
```

Just as we did in the previous section, we initialize a numeric string. PHP chooses the `string` type based on syntax, in this case the enclosing quote characters. Then we use the `int` data type in parentheses on the leftmost side of an assignment. Now we can see that `$num` is an integer. We can also do this on variable initialization:

```
$myNum = (string) 123;
$myNumStr = (int) "123";
$myFloatStr = (float) "1.23";
$myAlphaNumStr = (int) "123abc";

var_dump($myNum);
var_dump($myNumStr);
var_dump($myFloatStr);
var_dump($myAlphaNumStr);

Output:
string(3) "123"
int(123)
float(1.23)
int(123)
```

By putting the name of the desired type in parentheses before the literal value, we force the value to be cast to that type. `$myNum` would, without the `(string)`, otherwise be an `int`, `$myNumStr` likewise would have been a `string`. Pay attention to `$myAlphaNumStr`, see how PHP has discarded the letters? All these type conversions have a set of rules that govern equivalence. This is particularly important when comparing values or testing for truthiness, which we explore in more detail in Chapter 4, "Logic and Control."

You are free to use type casting whenever a variable would be valid syntax in an expression. We've seen casting on initialization above; it can also be done during operations and function calls:

```
$expr = (string) 3 . ' is the magic number';
var_dump($expr);
var_dump((int) '123');
```